



**RV College of
Engineering®**

Go, change the world

Unit 2

Cortex M Architecture

MGRJ

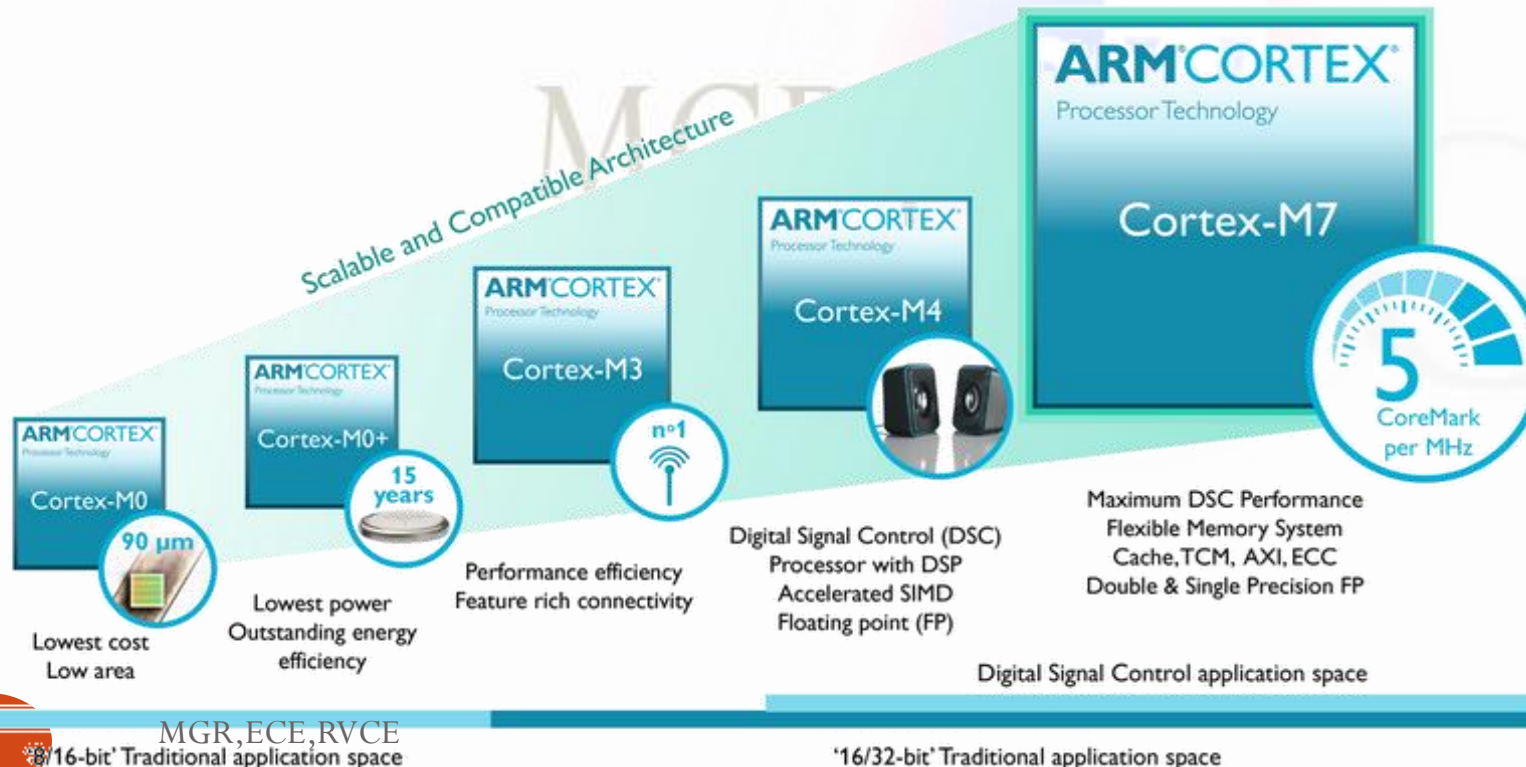
Unit 2: Syllabus

Cortex M Architecture:

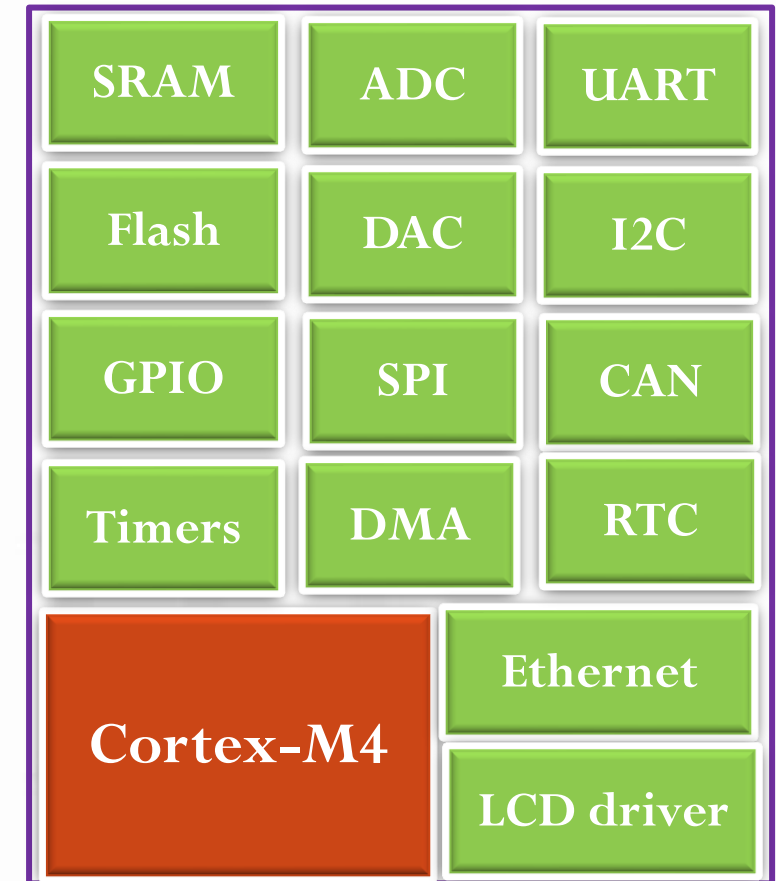
Advantages of Cortex M CPUs, Programmer's model: Operation modes & states, Registers, Special Registers, APSR, Memory System, Low power modes, Instruction Set: Memory access instructions, Arithmetic, Logical, Shift, Program flow control instructions, Programming examples, IDEs, ST-Link debugger.

ARM Cortex- M processors

- The cortex M processors use a 32-bit architecture. Internal registers in the register bank, the data path, and the bus interfaces are all 32 bits wide.
- The Instruction Set Architecture (ISA) in the Cortex-M processors is called the Thumb ISA and is based on Thumb-2 Technology which supports a mixture of 16-bit and 32-bit instructions.



STM32F407VG



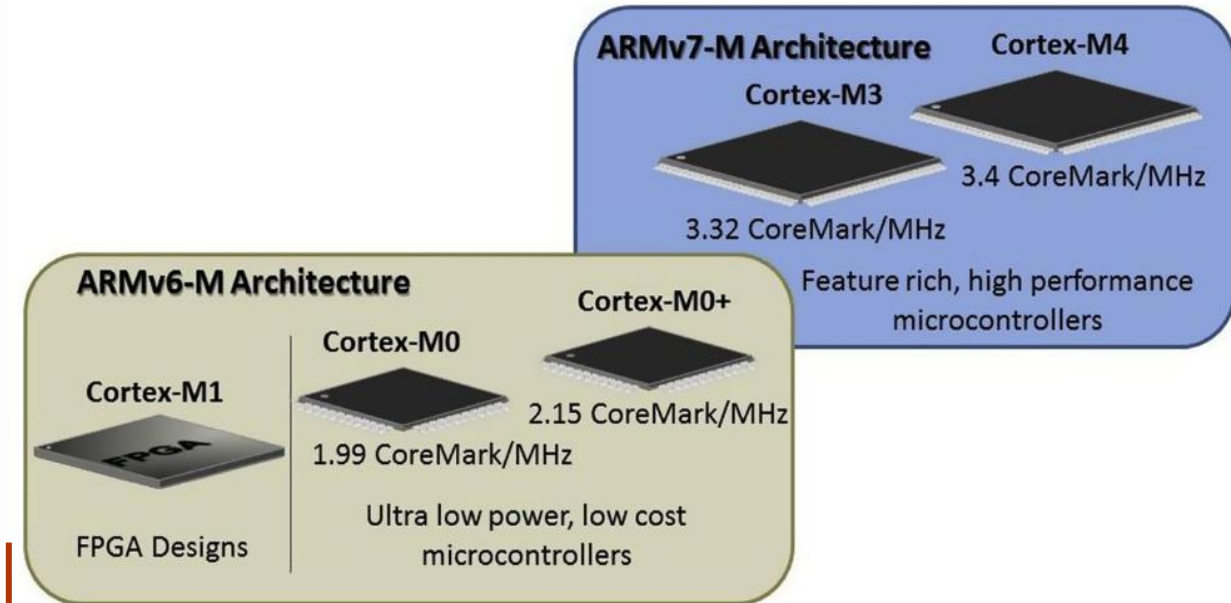
- Cortex M4 IP is provided by arm, others are put by STMicroelectronics

Features of Cortex- M4 processors

- Three-stage pipeline design.
- Harvard bus architecture with unified memory space: instructions and data use the same address space
- 32-bit addressing, supporting 4GB of memory space.
- On-chip bus interfaces based on ARM AMBA (Advanced Microcontroller Bus Architecture) Technology.
- An interrupt controller called NVIC (Nested Vectored Interrupt Controller) supporting up to 240 interrupt requests and from 8 to 256 interrupt priority levels.
- Support for various features for OS (Operating System) implementation.
- Sleep mode support and many low power features
- Instruction types: Data processing, Memory transfer, multiply & accumulate, Branches, OS support, Single instruction multiple data(SIMD),SP floating point, etc.

Cortex M family revisit

- The Cortex-M3 and Cortex-M4 processors are based on ARMv7-M architecture.
- Both are high-performance processors that are designed for microcontrollers.
- Cortex-M4 processor can also carry out some of the digital signal processing applications that traditionally have been carried out by a separate Digital Signal Processor.
- The Cortex-M0, Cortex-M0+, and the Cortex-M1 processors are based on ARMv6-M, which has a smaller instruction set.



CoreMark Rating Samples:

AMD Ryzen 9:224 CoreMark/MHz

Intel Core i7-8700:11.19 CoreMark/MHz

STM32F446RE:3.51 CoreMark/MHz

Youtube link: Porting CoreMark to embedded platform

<https://www.youtube.com/watch?v=V-lglWXxadE&t=7s>

Advantages of Cortex M Processors

Low Power

- Many Cortex M microcontroller have power consumption less than 100uA/MHz.
- Architecture is supported with extensive low power modes.

E.g: *NXP LPC54113 Cortex-M4 MCU*: Execution from RAM as low as 81 $\mu\text{A}/\text{MHz}$ for Cortex-M4F.

NXP KL27Z Cortex-M0 MCU : 1.96 μA in deep sleep mode (RAM + RTC retained) and 93 μs wakeup time.

STM32F407xxx Cortex M4 MCU: 500 mw typical Power consumption.

Performance

- The Cortex M processors can deliver over 3 CoreMark/MHz and 1.25 DMIPS/MHz.

Advantages of Cortex M Processors...

-Code density

- The Thumb ISA provides excellent code density.
- Reduces power consumption, Cost and size.

-Interrupts

- The Cortex-M3 and Cortex-M4 processors have a configurable interrupt controller design, which can support up to 240 vectored interrupts and multiple levels of interrupt priorities.
- Interrupt latency is only 12 clock cycles for systems with zero wait state memory.

-Ease of use, C friendly

- The Cortex-M processors are very easy to use. In fact, they are easier than compared to many 8-bit processors because Cortex-M processors have a simple, **linear memory map**, and there are no special architectural restrictions, which you often find in 8-bit microcontrollers.

Advantages of Cortex M Processors...

-Energy Efficiency

- With limited supply energy, lot of processing can be done.
- Doing job faster and allow the system to stay in sleep modes for longer duration to enable battery operated applications.

E.g: Cortex M7 can deliver up to 6.5 CoreMark/mw at 1.8 V.

-Scalability/Broad Product portfolio

- The Cortex-M processor family allows easy scaling of designs from low-cost, simple microcontrollers costing less than a dollar to high-end microcontrollers running at 200 MHz or more.

-Debug features

- Besides standard design features, which we can find in most microcontrollers like halting and single stepping, we can also generate a trace to capture program flow, data changes, profiling information, and so on.

Advantages of Cortex M Processors...

-OS support

- A number of features are available to make OS implementation easier and make OS operations more efficient.

-Software portability and reusability

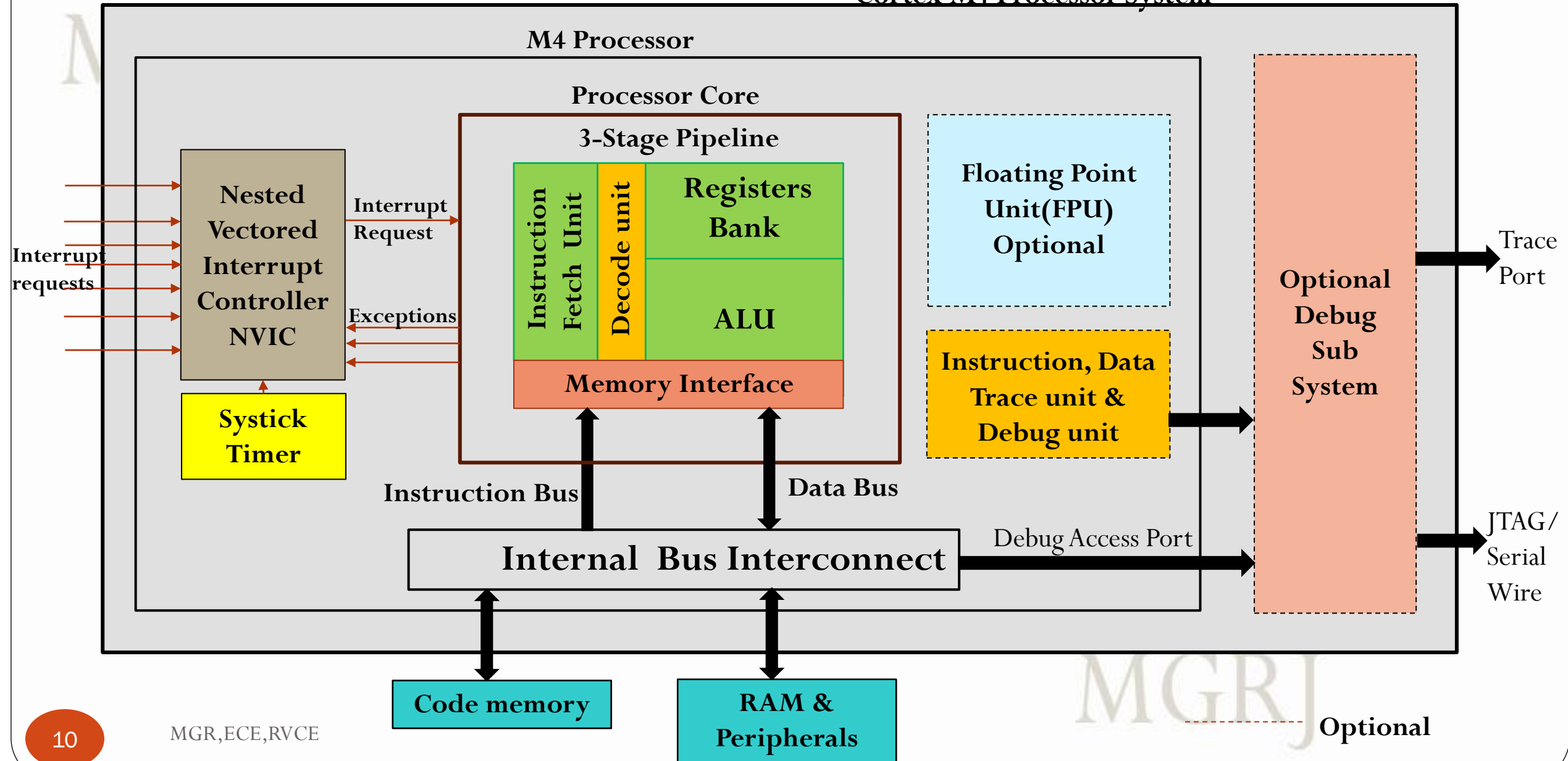
- One of ARM's initiatives called CMSIS (Cortex Microcontroller Software Interface Standard) makes programming for Cortex-M processor based products easier by providing **standard header files and an API** for standard Cortex-M processor functions.

-Choices (Devices, Tools, OS, etc.)

- One of the best things about using Cortex-M microcontrollers number amount of available choices.

Cortex M-4 CPU Architecture: Block Diagram

Cortex M4 Processor System



Cortex M-4 CPU Architecture:

- The Cortex-M4 processors contain the core of the processor, the Nested Vectored Interrupt Controller (NVIC), the SysTick timer, and optionally the floating point unit (for Cortex-M4).
- Apart from these, the processors also contain some internal bus systems and a set of components to support debug operations.
- The processor core consists of 3-stage pipeline with fetch unit, decode unit and execution unit shown as register bank and ALU with separate buses for instruction fetch and data access.
- NVIC support up to 240 peripheral interrupts and exceptions 1 to 15 are classified as system exceptions, and exceptions 16 and above are for interrupts.
- NVIC does priority based servicing, takes care of masking, pending requests, vector location for different interrupts.
- SysTick is a 24-bit timer designed mainly for generating regular OS interrupts. It can also be used by application code if OS is not in use.

Cortex M-4 CPU Architecture:

- Floating Point Unit(FPU) if present, perform operations in single precision on rational numbers. FPU is supported with separate registers and instruction set.
- The debug unit enables us to: stop program execution, examine and alter processor, examine and alter memory and input/output peripheral state.
- Instruction trace feature allows user to record of the instructions executed by the processor, enabling developers to analyze and debug the program flow at an instruction-level.
- Data trace allows the user to collect and analyze the data-related information during program execution especially memory read and write operations.
- A trace port, is a hardware interface used to capture and transmit trace data from a processor during program execution.
- JTAG (Joint Test Action Group) is a physical interface that allows for programming, debugging, and testing of these devices.

Programmer's Model:

Operating states and modes

- The Cortex-M4 processors have two **operation states and two modes**.
- In addition, the processors can have **privileged and unprivileged access levels**.

Operating states

- **Debug state:** When the processor is halted (e.g., by the debugger, or after hitting a breakpoint), it enters debug state and stops executing instructions.
- **Thumb state:** If the processor is running program code (Thumb instructions), it is in the Thumb state.

Operation modes (Applicable to Thumb state)

- **Handler mode:** Used to execute an exception handler such as an Interrupt Service Routine (ISR). When in handler mode, the processor always has privileged access level.
- **Thread mode:** When executing normal application code, the processor can be either in privileged access level or unprivileged access level.

Application code: LED toggling code With ISR

```
#include "main.h"
int main(void){
    HAL_Init();
    SystemClock_Config();
    MX_GPIO_Init();
    while (1)
    {
        HAL_GPIO_WritePin(GPIOD,GPIO_PIN_15,GPIO_PIN_SET);
        HAL_Delay(1000);
        HAL_GPIO_WritePin(GPIOD,GPIO_PIN_15,GPIO_PIN_RESET);
        HAL_Delay(1000);
    }
}
```

**I am in Thread
mode
(Privileged/
unprivileged)**

Interrupt service routine/Exception Handler:

```
void ISR_Callback(uint16_t GPIO_Pin)
{
    HAL_GPIO_WritePin(GPIOD,GPIO_PIN_14,GPIO_PIN_SET);
    HAL_Delay(1000);
    HAL_GPIO_WritePin(GPIOD,GPIO_PIN_14,GPIO_PIN_RESET);
    HAL_Delay(1000);
}
```

**I am in
Handler
mode
(Privileged)**

Privileged Vs unprivileged access level

- The *privileged access level* can access all resources in the processor, while *unprivileged access level* means some memory regions(protected) are inaccessible, and a few operations cannot be used.
- Software can switch the processor in privileged Thread mode to unprivileged Thread mode. However, it cannot switch itself back from unprivileged to privileged. This is controlled by a special register called “**CONTROL**.”

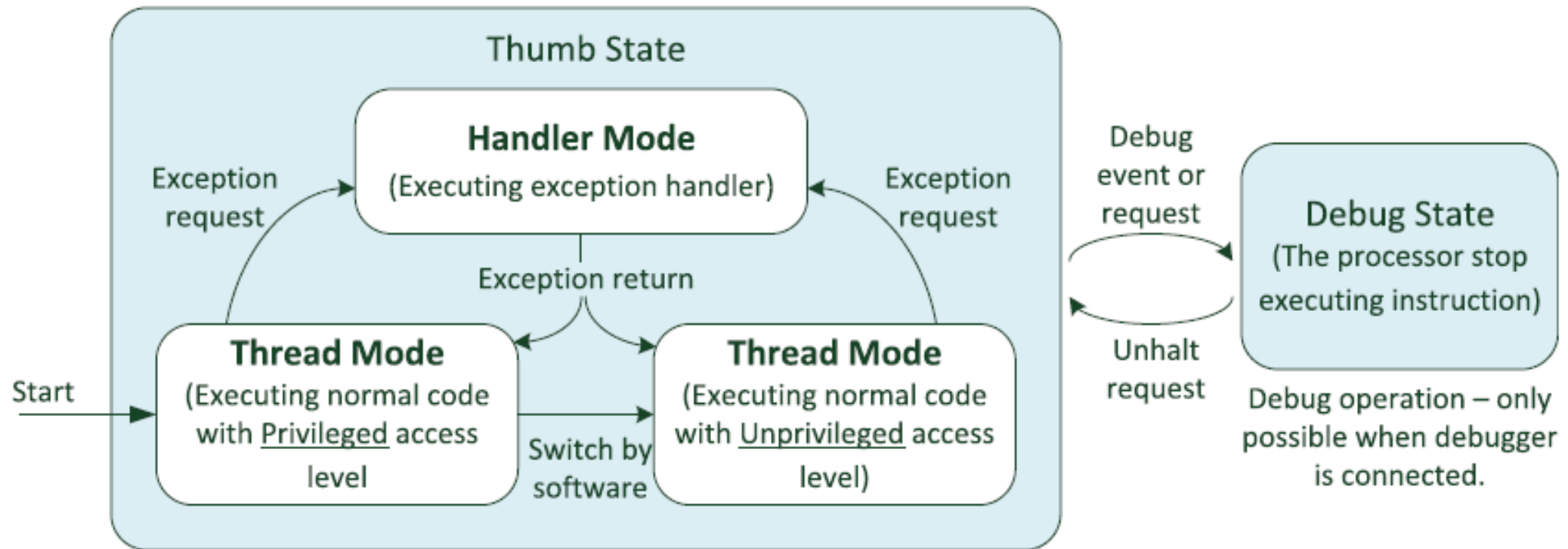


Privileged
Access all
resources
(Memory regions
and Registers)



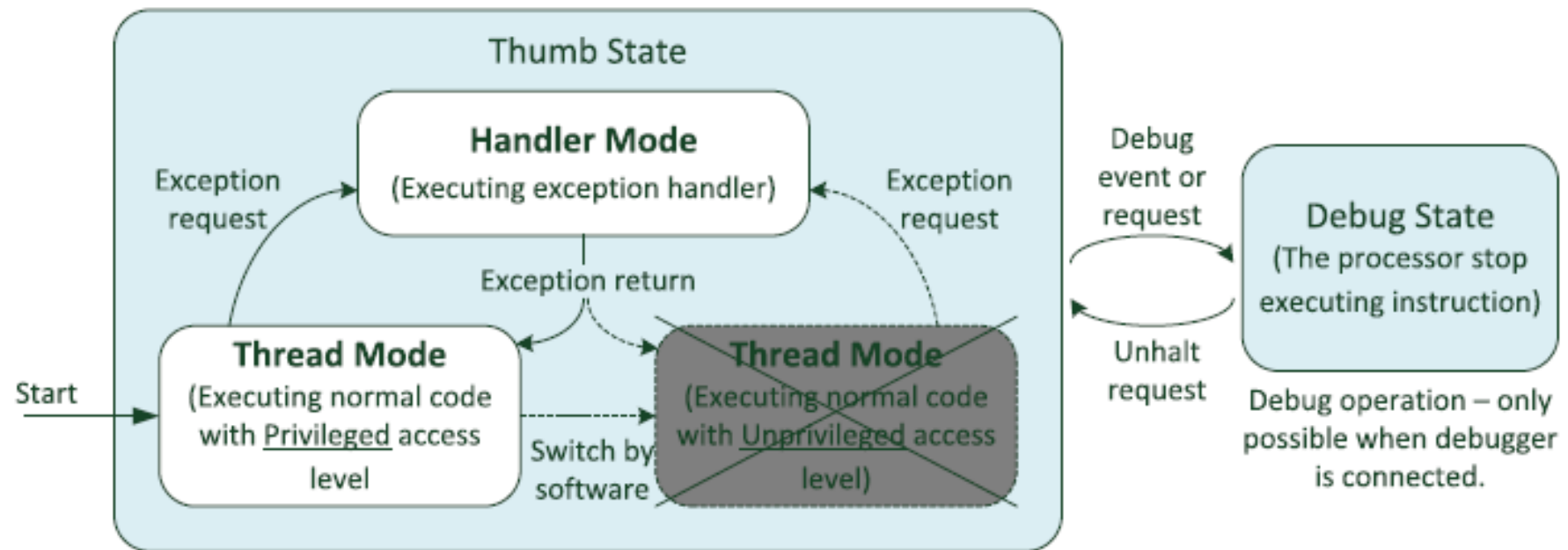
Unprivileged
No access to
protected memory
region and
CONTROL, NVIC
registers

Operating states and modes



Programmer's Model....

- By default, the Cortex-M processors start in privileged Thread mode and in Thumb state.
- In many simple applications, there is no need to use the unprivileged Thread mode.



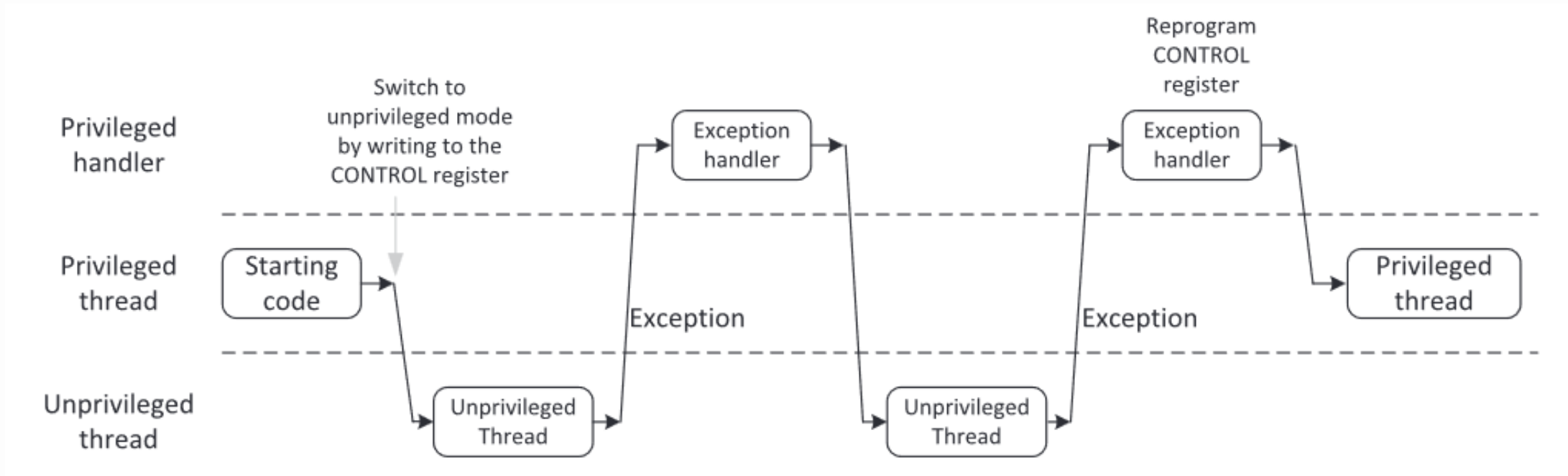
Use of privileged and unprivileged access levels

- The separation of access levels allow system designers to develop **robust embedded systems** by providing a mechanism to safeguard memory accesses to critical regions and by providing a basic security model.
- This is particularly important in systems which are developed on some embedded OS like FreeRTOS. The OS operations are called as **system tasks** and user defined operations are called as **application tasks(User tasks)**.
- The **application tasks** are usually made unprivileged to avoid corrupting of memory(critical region) used by **system tasks**. If an application task **crashes(fails)**, the remaining application tasks and the OS can still continue to run.
- In addition to above, the Cortex M4 processor system may be supported with **Memory Protection Unit(MPU)** to provide define memory access permission to different regions.

MGRJ

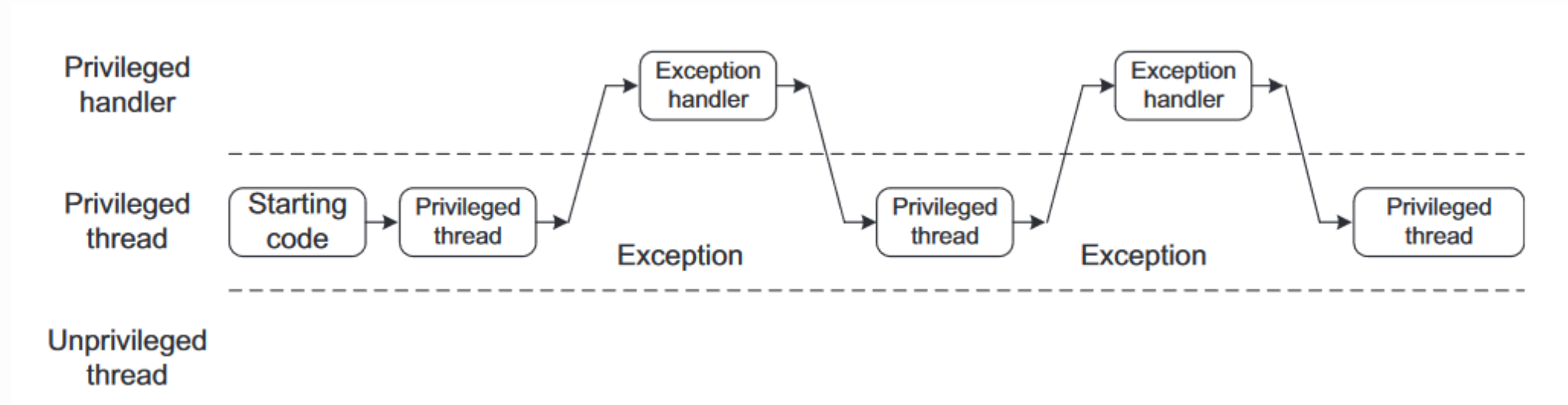
**Demo of operating states/modes/privileged with
Experiment 1 in Keil IDE**

Switching between privileged thread mode and unprivileged thread mode



MGRJ

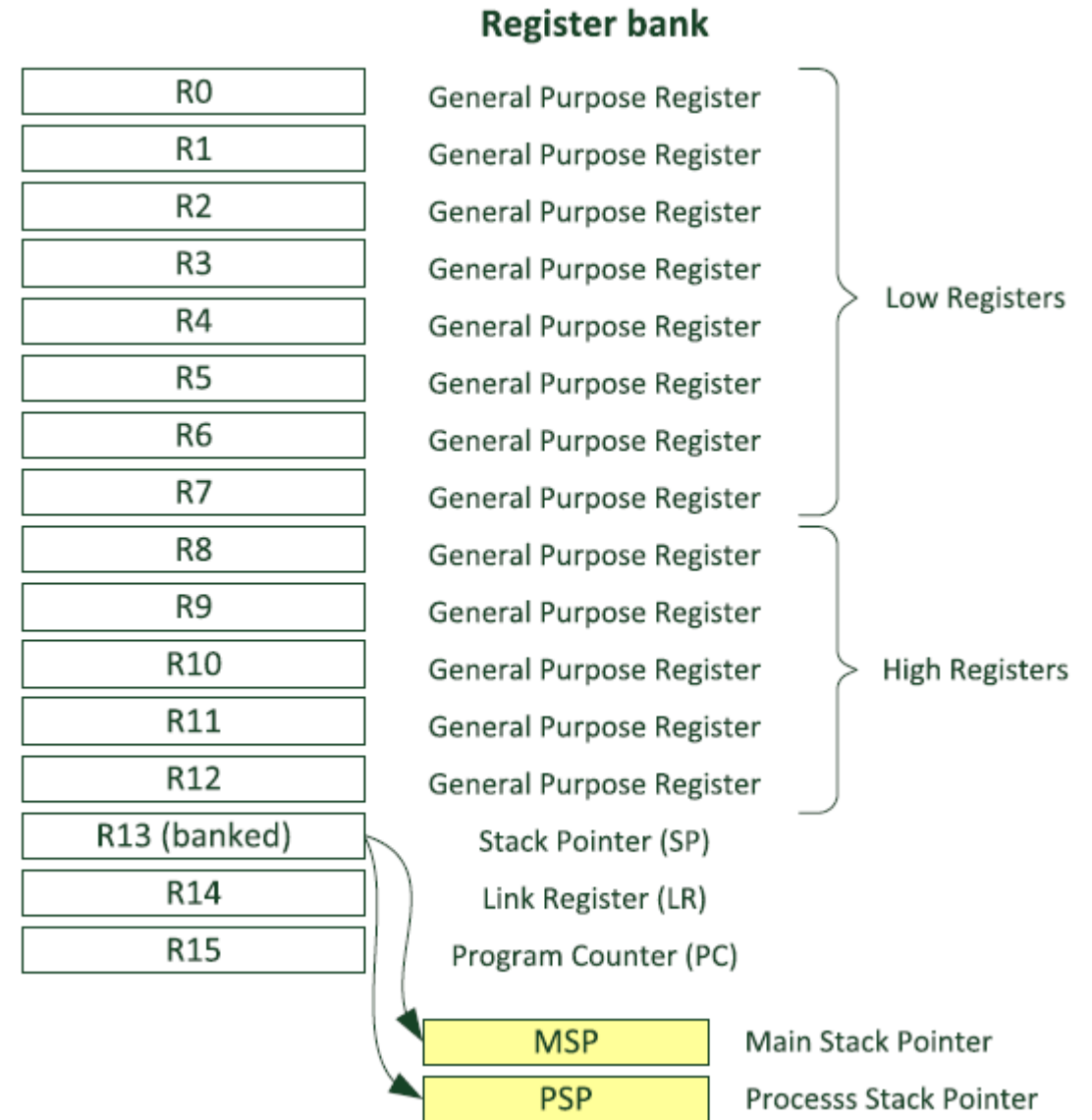
Simple applications do not require unprivileged Thread mode



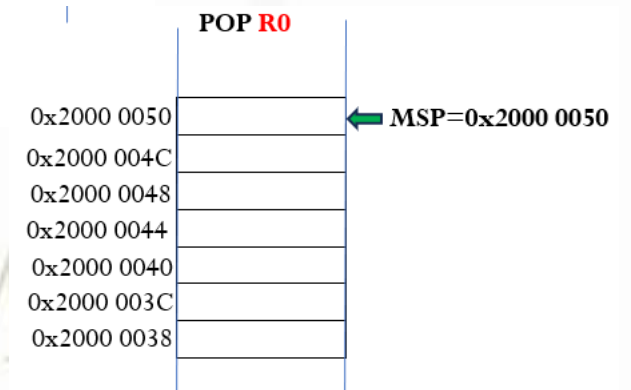
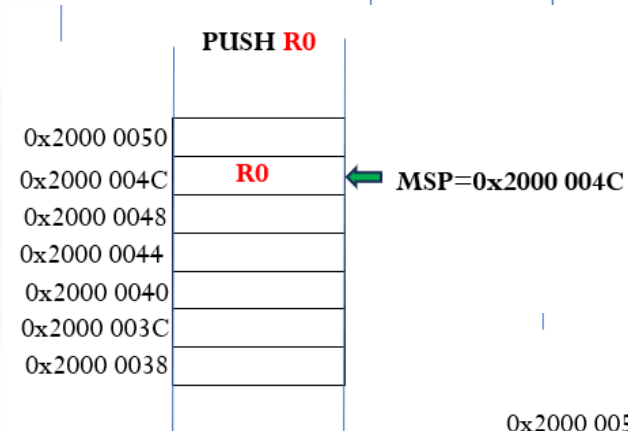
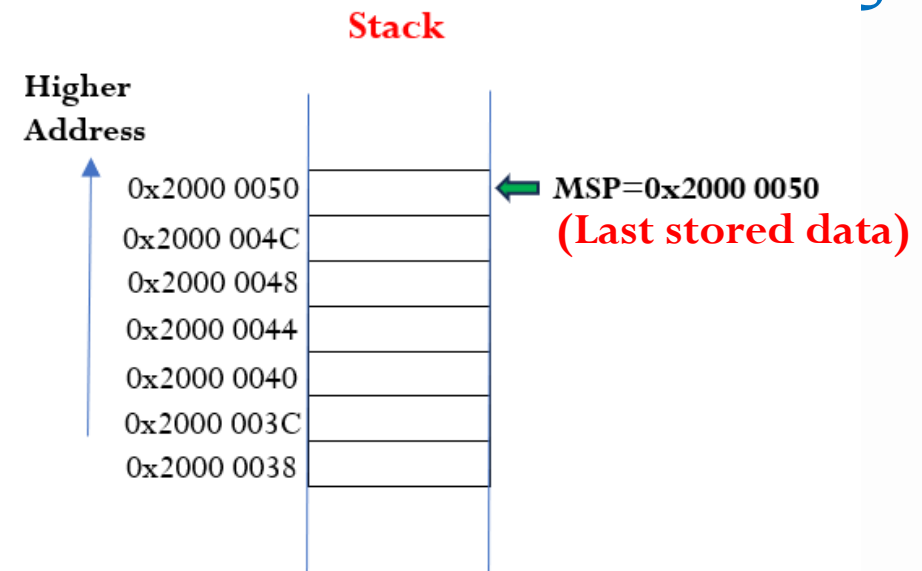
MGRJ

Register Bank

- The first eight (R0 -R7) are also called **low registers**. These registers can be accessed by all instructions(32 bit and 16 bit)
- The **high registers** (R8 -R12)can be used with 32-bit instructions only.
(Why is this constraint?)
- ARM is **load-store architecture** so that, if data in memory is to be processed, it has to be loaded from the memory to registers.



- **Stack** is a portion of data memory used as temporary data storage and pointer to stack is called **Stack Pointer** and stack always works in Last in First Out(LIFO) order.
- **R13 is the Stack Pointer(SP)** used for accessing the stack memory via **PUSH**(Storing data on stack) and **POP**(Retrieving data from stack) operations.
- Physically there are two different Stack Pointers: **the Main Stack Pointer(MSP)** is the default Stack Pointer. It is selected after reset, or when the processor is in Handler Mode.
- The other Stack Pointer is called the **Process Stack Pointer (PSP)** normally used when an embedded OS is involved, where the stack for the OS kernel(PSP) and application tasks(MSP) are separated.
- The selection of Stack Pointer is determined by a special register called **CONTROL**.



Registers...

- In ARM Cortex-M processors, PUSH and POP are always 32-bit, and the addresses of the transfers in stack operations must be aligned to 32-bit(4 byte) word boundaries.
- Note that, there are no PUSH and POP mnemonic instead STM(store multiple) and LDM(Load multiple) are used respectively.
- Identify the use of following program statements:

PRESERVE8

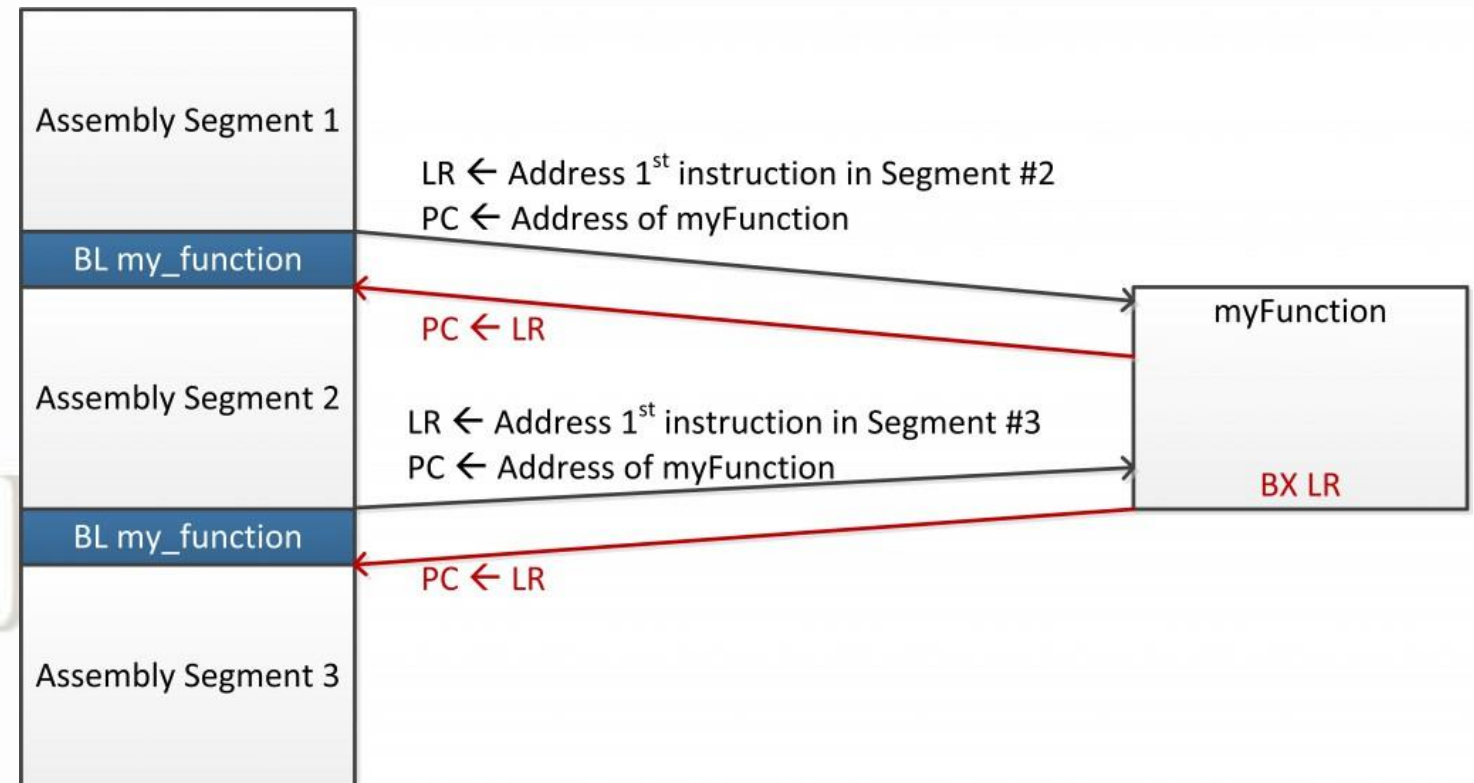
Stack EQU 0x0000 0100

AREA STACK, NOINIT, READWRITE, ALIGN=3

StackMem SPACE Stack

Registers...

- R14 is also called the **Link Register (LR)**. This is used for holding the return address when calling a function or subroutine.
- At the end of the function or subroutine, the program control can return to the calling program and resume by loading the value of LR into the Program Counter (PC).
- R15 is the **Program Counter (PC)** holds the address of next instruction to be executed.



Source: UNIVERSITY of WISCONSIN-MADISON

- **Processor status registers**

- Application Program Status register(APSR)
- Execution Program Status register(EPSR)
- Interrupt Program Status register(IPSr)

- **Interrupt/Exception masking registers**

- PRIMASK register
- FAULTMASK register
- BASEPRI register

Discussed in last unit

- **Operation State(access level) control register**

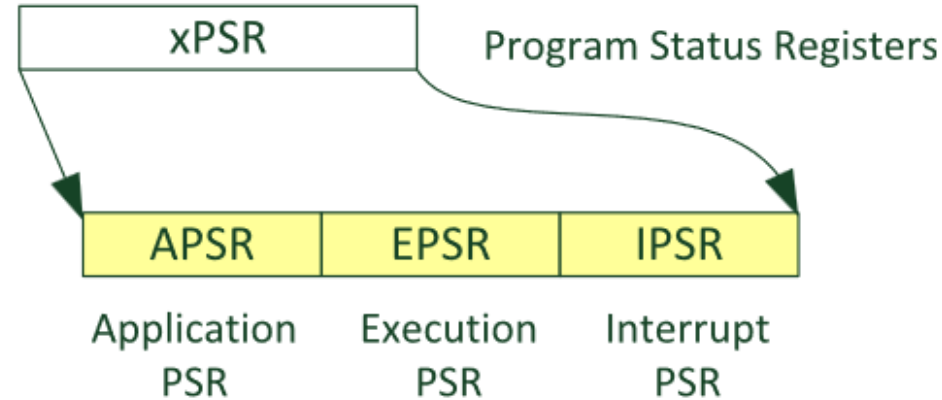
- CONTROL register

Note: Special registers are not memory mapped, and can be accessed using special register access instructions such as MSR(write to special register) and MRS(Read).

MGR,ECE,RVCE

Program Status Registers

- The three registers can be accessed using xPSR.



- In ARM assembler, when accessing xPSR, the symbol PSR is used.

For example:

MRS r0, PSR ; Read the combined program status word

MSR PSR, r0 ; Write combined program state word

Program Status Registers...

- Bit fields:

	31	30	29	28	27	26:25	24	23:20	19:16	15:10	9	8	7	6	5	4:0
APSR	N	Z	C	V	Q				GE*							
IPSR												Exception Number				
EPSR						ICI/IT	T				ICI/IT					

N Negative flag

Z Zero flag

C Carry (or NOT borrow) flag

V Overflow flag

Q Sticky saturation flag

GE[3:0] Greater-Than or Equal flags for SIMD instruction.

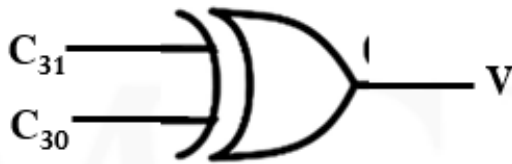
ICI/IT Interrupt-Continuable Instruction (ICI) bits,
IF-THEN instruction status bit for conditional execution

T Thumb state, always 1

Exception Number indicate which exception the processor is handling.

ALU Flags

- **Carry Flag(C):** Set to 1 if carry generated for 31st bit addition. For subtract operations, this bit is the inverse of the borrow output status.
- **Negative Flag(N):** Set to 31st bit of the result of the executed instruction.
- **Zero flag(Z):** Set to 1 if the result of the executed instruction is zero.
- **Overflow flag(V):** Overflow of the result.



C_{31} : Carry out of 31 bit addition(borrow in the case of subtraction).

C_{30} : Carry out of 30 bit addition(borrow in the case of subtraction).

Examples:

Operation	Result	Flag
$0x70000000 + 0x70000000$	$0xE0000000$	$N=1, Z=0, C=0, V=1$
$0x90000000 + 0x90000000$	$0x20000000$	$N=0, Z=0, C=1, V=1$
$0x00001234 - 0x00001000$	$0x00000234$	$N=0, Z=0, C=1, V=0$
$0x00000004 - 0x00000005$	$0xFFFFFFFF$	$N=1, Z=0, C=0, V=0$
$0x80000005 - 0x80000004$	$0x00000001$	$N=0, Z=0, C=1, V=0$
$0x70000000 - 0xF0000000$	$0x80000000$	$N=1, Z=0, C=0, V=1$
$0xA0000000 - 0xA0000000$	$0x00000000$	$N=0, Z=1, C=1, V=0$

CONTROL register

- This register defines the following:
 - The selection of stack pointer (Main Stack Pointer/Process Stack Pointer)
 - Access level in Thread mode (Privileged/Unprivileged)

	31:3	2	1	0
CONTROL			SPSEL	nPRIV

nPRIV (bit 0):

Defines the privileged level in Thread mode

- When this bit is 0 (default), it is privileged level when in Thread mode.
- When this bit is 1, it is unprivileged when in Thread mode.

SPSEL (bit 1):

Defines the Stack Pointer selection:

- 0 (default), Thread mode uses MSP.
- When this bit is 1, Thread mode uses PSP.
- In Handler mode, this bit is always 0 and write to this bit is ignored.

Questions

1. What is the status of nPRIV bit in handler mode?
2. If nPRIV=1 thread mode, how to change mode to privileged?
3. Identify the operation(use scenario) of following combinations of SPSEL and nPRIV bits?

nPRIV	SPSEL	Use Case
0	0	The current executing main program in privileged with MSP. Exception handler using MSP.
0	1	The current executing main program in privileged with PSP. Exception handler using MSP.
1	1	The current executing main program in unprivileged with PSP. Exception handler using MSP.
1	0	The current executing program is unprivileged using MSP. Exception handler using MSP.

Commonly used directives

• Defining data (arm assembler in Keil):

Byte (8 bits)	DCB	E.g: value DCB 0x12
Half words (16 bits)	DCW	E.g: value2 DCW 0x1234
Word (32 bits)	DCD	E.g: value3 DCD 0x1234 5678
Double word(64 bits)	DCQ	E.g: value4 DCQ 0x12345678 FFAB CDEF
SP Float(32 bits)	DCFS	E.g: value5 DCFS 1.234
DP float(64 bits)	DCFD	E.g: value6 FCFD 1.341956
String	DCB	E.g: name DCB "Katrina\n",0 ← Null for string termination

Equivalent C declarations(arm C compiler in Keil):

DCB	char(signed)	DCFS	float
DCW	short(signed)	DCFD	double/long double
DCD	int/long(signed)		
DCQ	long long(signed)		

DCB: Define Constant Byte, DCD:Define Constant Data

- Identify the operation of following code snippets:

Listing 1:

```
MOVS R0, #0x12          ; Set R0 = 0x12 (hexadecimal)
MOV R1, #'A'            ; Set R1 = ASCII character A
```

Listing 2:

```
NVIC_IRQ_SETEN EQU 0xE000E100
NVIC_IRQ0_ENABLE EQU 0x1
```

```
LDR R0,=NVIC_IRQ_SETEN    ; Put 0xE000E100 into R0
MOV R1, #NVIC_IRQ0_ENABLE ; Put immediate data (0x1) into
STR R1, [R0]              ; Store 0x1 to 0xE000E100,
```

- Identify the operation of following code snippet:

Listing 3:

LDR R3,=MY_NUMBER ; Get the memory location of MY_NUMBER

LDR R4, [R3] ; Read the value 0x12345678 into R4

LDR R0,=HELLO_TEXT ; Get the starting address of HELLO_TEXT

BL PrintText ; Call a function called PrintText to display string

MY_NUMBER DCD 0x12345678

HELLO_TEXT DCB "Hello\n", 0 ; Null terminated string

Instruction set

- Instructions for Moving data within Processor

Instruction	Destination	Source	Operation
MOV	R4,	R0	Copy value from R0 to R4
MOVS	R4,	R0	Copy value from R0 to R4 with APSR update
MRS	R7,	PRIMASK	Copy value of PRIMASK to R7
MSR	CONTROL,	R2	Copy value of R2 into CONTROL
MOV	R3,	#0x34	Set R3 value to 0x34
MOVS	R3,	#0x34	Set R3 value to 0x34 with APSR update
MOVW	R6,	#0x1234	Set R6 to a 16-bit constant 0x1234
MOVT	R6,	#0x8765	Set the upper 16-bit of R6 to 0x8765
MVN	R3,	R7	Move negative value of R7 into R3

- MOV/MOVS are used to set a register to 8 bit immediate value.
- MOVW is used to set a register to 16 bit value.
- **To set a register to 32 bit value LDR is used:**

LDR R0,=0x12345678

- This is not a real instruction. The assembler converts this instruction into a memory transfer instruction and a literal data item stored in the program memory.

LDR R0,[PC,#offset]

.....

.....

DCD 0x12345678

MOV instruction...

- Program listing in armasm in Keil(Cortex M4 is 23 stage pipeline)

```
MOV R0,#0x34
MOVW R1,#1234
MOVT R2,#0x1234
LDR R4,=0xFFFFEEEE
NOP
NOP
NOP

STOP B STOP

END
```

Disassembly

39:	MOV R0,#0x34		
0x08000010	F04F0034	MOV	r0,#0x34
40:	MOVW R1,#1234		
0x08000014	F24041D2	MOVW	r1,#0x4D2
41:	MOVT R2,#0x1234		
0x08000018	F2C12234	MOVT	r2,#0x1234
42:	LDR R4,=0xFFFFEEEE		
0x0800001C	4C02	LDR	r4,[pc,#8] ; @0x08000028
43:	NOP		
0x0800001E	BF00	NOP	← Decode(PC-2)
44:	NOP		
0x08000020	BF00	NOP	← Fetch (PC)
45:	NOP		
46:			
0x08000022	BF00	NOP	← PC+2
47: STOP B STOP			
0x08000024	E7FE	B	← PC+4
0x08000026	0000	DCW	← PC+6
0x08000028	EEEE	DCW	← PC+8
0x0800002A	FFFF	DCW	

Zero Padding

- Memory Access instructions

Data type	Load(Read from memory)	Store(Write to memory)
8 bit unsigned	LDRB(Zero extend)	STRB
8 bit signed	LDRSB(Sign extend)	STRB
16 bit unsigned	LDRH(Zero Extend)	STRH
16 bit signed	LDRSH(Sign Extend)	STRH
32 bit	LDR	STR
Multiple 32 bit	LDM	STM
Double word(64 bit)	LDRD	STRD

• Zero Extend

Before Execution:

mem[0x2000 0000]=0x83

R1=0x2000 0000

R0=0xFFFF FF00

LDRB R0,[R1]

After Execution:

mem[0x2000 0000]=0x83

R1=0x2000 0000

R0=0x0000 0083

Before Execution:

mem[0x2000 0000]=0x9083

R1=0x2000 0000

R0=0xFFFF FF00

LDRH R0,[R1]

After Execution:

mem[0x2000 0000]=0x9083

R1=0x2000 0000

R0=0x0000 9083

- **Sign Extend**

Before Execution:

mem[0x2000 0000]=0x83

R1=0x2000 0000

R0=0x0000 00F0

LDRSB R0,[R1]

After Execution:

mem[0x2000 0000]=0x83

R1=0x2000 0000

R0=0xFFFF FF83

Before Execution:

mem[0x2000 0000]=0x9083

R1=0x2000 0000

R0=0x0000 FFFF

LDRSH R0,[R1]

After Execution:

mem[0x2000 0000]=0x9083

R1=0x2000 0000

R0=0xFFFF 9083

Memory Access instructions with immediate offset: Pre-index

Pre Index	Description
LDRB Rd,[Rn,#offset]	Read byte from memory location Rn+offset
LDRSB Rd,[Rn,#offset]	Read byte and sign extend from memory location Rn+offset
LDRH Rd,[Rn,#offset]	Read halfword from memory location Rn+offset
LDRSH Rd,[Rn,#offset]	Read halfword and sign extend from memory location Rn+offset
LDR Rd,[Rn,#offset]	Read word from memory location Rn+offset
LDRD Rd1,Rd2[Rn,#offset]	Read double-word from memory location Rn + offset
STRB Rd,[Rn,#offset]	Store byte to memory location Rn+offset
STRH Rd,[Rn,#offset]	Store Halfword to memory location Rn+offset
STR Rd,[Rn,#offset]	Store word to memory location Rn+offset
STRD Rd1,Rd2,[Rn,#offset]	Store double word to memory location Rn+offset

- **Pre-index Examples**

Before Execution:

mem[0x2000 0000]=0x83

mem[0x2000 0004]=0xFF

R1=0x2000 0000

R0=0xFFFF FF00

LDRB R0,[R1,#4]

After Execution:

mem[0x2000 0000]=0x83

mem[0x2000 0004]=0xFF

R1=0x2000 0000

R0=0x0000 00FF

Before Execution:

mem[0x2000 0000]=0x83

mem[0x2000 0004]=0xaabbccdd

R1=0x2000 0000

R0=0x12345678

STR R0,[R1,#4]

After Execution:

mem[0x2000 0000]=0x83

mem[0x2000 0004]=0x12345678

R1=0x2000 0000

R0=0x12345678

• Pre-index Examples

Before Execution:

mem[0x2000 0000]=0x83

mem[0x2000 0004]=0xFF

R1=0x2000 0008

R2=0xFFFF FF00

R3=0x3457AABB

LDRD R2,R3, [R1,#-4]

After Execution:

mem[0x2000 0000]=0x83

mem[0x2000 0004]=0xFF

R1=0x2000 0004

R2=0x0000 0083

R3=0x0000 00FF

Before Execution:

mem[0x2000 0000]=0x83

mem[0x2000 0004]=0xaabbccdd

R1=0x2000 0008

R2=0xFFFF FF00

R3=0x3457AABB

STRD R2,R3, [R1,#-4]

After Execution:

mem[0x2000 0000]=0xFFFF FF00

mem[0x2000 0004]=0x3457AABB

R1=0x2000 0008

R2=0xFFFF FF00

R3=0x3457AABB

- Offset must be always word aligned

Pre-index with address register update

- The **exclamation mark (!)** in the instruction specifies whether the register holding the address should be updated (write back) when the instruction is completed.

Instruction	Operation
LDR R0, [R1, #0x8]!	R0= Read from memory[R1+8] R1= R1+8
STRD R2,R3, [R1,#-8]!	Store R2,R3 at memory [R1-8] R1= R1-8

Pre-index with register offset

Instruction	Operation
LDR R5, [R0,R7]	R5= Read from memory[R0+R7]
STR R2, [R1,R4]!	Store R2 at memory [R1+R4] R1= R1+R4
LDR r0,[r1,r2,LSL#2]!	r0=Read from memory[r1+(r2<<2)] r1=r1+(r2<<2)
LDR r0,[r1,r2,LSL #0x2]	r0=Read from memory[r1+(r2<<2)]

- LSL: Logical Shift left (only LSL is allowed)
- Shift range is 0 to 3.

Questions

- Identify the operation of following instructions:

i. **MOV R2,R3, LSL #4**

ii. **MOV R2,R3,ASR R1**

iii. **MOV R2,R3, LSR R1**

- Write a program segment to transfer 32 bit data starting at 0x2000 0010 to a location 0x2000 0024.

LDR R0,= 0x2000 0010

LDR R1,= 0x2000 0024

LDR R2,[R0]

STR R2,[R1]

- Write the code snippet to access the contents of the array **myDat** and store in **newDat** declared as follows:

```
AREA |.data|,DATA,READWRITE,ALIGN=3
```

```
myDat DCW 1,2,3,4,5,6,
```

```
newDat SPACE 12
```

```
MOV R4,#6
```

```
MOV R3,#0
```

```
LDR R0,= myDat
```

```
LDR R1,= newDat
```

repeat

```
LDRH R2, [R0,R3,LSL #1]
```

```
STRH R2,[R1, R3,LSL #1]
```

```
ADD R3,R3,#1
```

```
SUBS R4,R4,#1
```

```
BNE repeat
```

Memory Access instructions with immediate offset: **Post-index**

Pre Index	Description
LDRB Rd,[Rn],#offset	Rd= Read byte at [Rn], Rn=Rn+offset
LDRSB Rd,[Rn],#offset	Rd=sign extended byte at[Rn],Rn=Rn+offset
LDRH Rd,[Rn],#offset	Rd= Read halfword at [Rn], Rn=Rn+offset
LDRSH Rd,[Rn],#offset	Rd=sign extended word at[Rn],Rn=Rn+offset
LDR Rd,[Rn],#offset	Rd= Read word at [Rn], Rn=Rn+offset
LDRD Rd1,Rd2[Rn],#offset	Rd= Read double word at [Rn], Rn=Rn+offset
STRB Rd,[Rn],#offset	[Rn]=LSB from Rd, Rn=Rn+offset
STRH Rd,[Rn],#offset	[Rn]=Lower 16 bits from Rd, Rn=Rn+offset
STR Rd,[Rn],#offset	[Rn]=store Rd, Rn=Rn+offset
STRD Rd1,Rd2,[Rn],#offset	[Rn]=double word from Rd1,Rd2, Rn=Rn+offset

Questions

- Identify the difference between following instructions:

i. **LDRGT r0, [r1, #4]!**

ii. **LDRGT r0, [r1, #4]**

iii. **LDRGT r0, [r1], #4**

- Write the program segment to transfer 32 bit data starting at 0x2000 0010 to a location 0x2000 0024.

```
LDR R0,= 0x2000 0010
```

```
;LDR R1,= 0x2000 0024
```

```
LDR R2,[R0],#0x14
```

```
STR R2,[R0]
```

- Write the code snippet to access the contents of the array **myDat** and store in **newDat** declared as follows:(use post-index)

```
AREA |.data|,DATA,READWRITE,ALIGN=3
```

```
myDat DCH 1,2,3,4,5,6,
```

```
newDat SPACE 12
```

```
MOV R4,#6
```

```
LDR R0,= myDat
```

```
LDR R1,= newDat
```

repeat

```
LDRH R2, [R0],#2
```

```
STRH R2,[R1],#2
```

```
SUBS R4,R4,#1
```

```
BNE repeat
```

Multiple load and Multiple store

Memory access instructions...

- The LDM (Load Multiple registers) and STM (Store Multiple registers) instructions only support 32-bit data.
- They support two types of pre-indexing:
 - **IA**: Increment address **A**fter each read/write
 - **DB**: Decrement address **B**efore each read/write

Instruction	Operation
LDMIA Rn,{reg list}	Read multiple words from memory location specified by Rn. Address Increment After (IA) each read.
LDMDB Rn,{reg list}	Read multiple words from memory location specified by Rn. Address Decrement Before (DB) each read.
STMIA Rn,{reg list}	Write multiple words to memory location specified by Rn. Address increment after each write.
STMDB Rn,{reg list}	Write multiple words to memory location specified by Rn. Address Decrement Before each write.

- **Example:**

Before execution:

mem[0x2000 0000]=0x83
mem[0x2000 0004]=0x1000 00FF
mem[0x2000 0008]=0xFFFF 00FF
mem[0x2000 000C]=0x8900 0011
R0=0x0000 0000, R1=0x0000 0000
R2=0x0000 0000, R3=0x0000 0000
R4=0x0000 0000

LDR R4,=0x20000000

LDMIA R4, {R0-R3}

After execution:

mem[0x2000 0000]=0x83
mem[0x2000 0004]=0x1000 00FF
mem[0x2000 0008]=0xFFFF 00FF
mem[0x2000 000C]=0x8900 0011

R0=0x0000 0083, R1= 0x1000 00FF
R2= 0xFFFF 00FF, R3= 0x8900 0011
R4=0x2000 0000

- **Example: LDM with register update**

Before execution:

mem[0x2000 0000]=0x83

mem[0x2000 0004]=0x1000 00FF

mem[0x2000 0008]=0xFFFF 00FF

mem[0x2000 000C]=0x8900 0011

R0=0x0000 0000, R1=0x0000 0000

R2=0x0000 0000, R3=0x0000 0000

R4=0x0000 0000

LDR R4,=0x20000010

LDMDB R4!, {R0-R3}

After execution:

mem[0x2000 0000]=0x83

mem[0x2000 0004]=0x1000 00FF

mem[0x2000 0008]=0xFFFF 00FF

mem[0x2000 000C]=0x8900 0011

R0=0x0000 0083, R1= 0x1000 00FF

R2= 0xFFFF 00FF, R3= 0x8900 0011

R4=0x2000 0000

Questions

- What is the content of r0 after execution of following instruction and identify the locations where registers are stored?

STMDB r0!,{r1-r3}

r0=r0-3x4

r3=r0-4 r2=r0-8 r1=r0-12

- Write an assembly instruction to load 4 words starting at memory location 0x80000000 from registers r0-r3? (Assume r9 contains the base address 0x80000000)

LDMIA r9,{r0,r1,r2,r3} or

LDMIA r9,{r0-r3}

Multiple loads and stores...

- Can the LDM instruction be conditional in ARM?

Yes

LDMEQIA r0!,{r1-r3} (Zero flag=1)

- How many registers can be loaded or stored using a single LDM or STM instruction?

All general purpose registers
STMDB R0,{R0-R12}

Arithmetic Instructions

Instruction set...

Instruction	Operation	‘W’ indicate wide to generate machine code of 32 bit with 12 bit immediate data. Some assemblers with 8 bit immediate data generate 16 bit machine code. No difference in <i>armasm</i> in Keil. It always generate 32 bit machine code
ADD Rd, Rn, Rm	$Rd = Rn + Rm$	
ADD Rd, Rn, #immed	$Rd = Rn + \#immed$	
ADC Rd, Rn, Rm	$Rd = Rn + Rm + \text{carry}$	
ADC Rd, #immed	$Rd = Rd + \#immed + \text{carry}$	
ADDW Rd, Rn, #immed	$Rd = Rn + \#immed$	
SUB Rd, Rn, Rm	$Rd = Rn - Rm$	
SUB Rd, #immed	$Rd = Rd - \#immed$	
SUB Rd, Rn, #immed	$Rd = Rn - \#immed$	
SBC Rd, Rn, #immed	$Rd = Rn - \#immed - \text{borrow}$	
SBC Rd, Rn, Rm	$Rd = Rn - Rm - \text{borrow}$	

*No flags updated because suffix ‘S’ is not there

* All instructions are conditional

Arithmetic Instructions...

Instruction	Operation
RSB Rd, Rn, #immed	$Rd = \#immed - Rn$ (reverse subtraction)
RSB Rd, Rn, Rm	$Rd = Rm - Rn$
MUL Rd, Rn, Rm	$Rd = Rn * Rm$
MLA Rd, Rn, Rm, Ra	$Rd = Ra + Rn * Rm$ (Multiply & accumulate)
MLS Rd, Rn, Rm, Ra	$Rd = Ra - Rn * Rm$ (Multiply & subtract)
SMULL RdLo, RdHi, Rn, Rm	$\{RdHi, RdLo\} = Rn * Rm$ (Signed multiply long)
SMLAL RdLo, RdHi, Rn, Rm	$\{RdHi, RdLo\} += Rn * Rm$ (Signed MLA long)
UMULL RdLo, RdHi, Rn, Rm	$\{RdHi, RdLo\} = Rn * Rm$ (Unsigned MUL long)
UMLAL RdLo, RdHi, Rn, Rm	$\{RdHi, RdLo\} += Rn * Rm$ (Unsigned MLA long)

* Only RSB is made conditional updating of flags

* All instructions are conditional

Questions

- Write the operation of following code snippet.

```
LDR R0,=0x12345678
```

```
LDR R1,=0xABCDEF00
```

```
UMULL R2,R3,R0,R1
```

After Execution:

R0 x R1=0xC37 9AAA 42D2 0800

R2=0x42D2 0800(Lower 32 bits)

R3=0x0C37 9AAA(Upper 32 bits)

Arithmetic Instructions...

- Write the operation of following code snippet.

```
LDR R0,=0xFFFFFFFF
```

```
LDR R1,=0xFFFFFFFF
```

```
LDR R3,=0x1234
```

```
LDR R2,=0x0000
```

```
UMLAL R2,R3,R0,R1
```

After Execution:

R0xR1= FFFF FFFE 0000 0001

+

0000 0000 0000 1234

[R3,R2]=FFFF FFFE 00001235

Questions

- Write the operation of following code snippet.

LDR R0,=0x1112222(+ve number)

LDR R1,=0x1112222(+ve number)

LDR R3,=0x0000

LDR R2,=0x0000

SMLAL R2,R3,R0,R1

After Execution:

R0 * R1 = 0x12369 D111 0C84(+ve number)

R2=0x D111 0C84(Lower 32 bits)

R3=0x0001 2369 (Upper 32 bits)

- Signed Multiplication rules:

- $+A * +B = +AB$

- $+A * -B = -AB$

- $-A * -B = +AB$

Questions

- Write the operation of following code snippet.

LDR R0,=0x81112222(-ve number)

LDR R1,=0x81112222(-ve number)

LDR R3,=0x0000

LDR R2,=0x0000

SMLAL R2,R3,R0,R1

After Execution:

$R0 * R1 = 2\text{'s complement}(0x81112222) *$

$2\text{'s complement}(0x81112222)$

$= 0x7EEEDDDE * 0x7EEEDDDE$

$= 0x3EF0\ 0147\ D111\ 0C84$

$R2 = 0x\ D111\ 0C84$ (Lower 32 bits)

$R3 = 0x\ 3EF0\ 0147$ (Upper 32 bits)

- Write the operation of following code snippet.

LDR R0,=0x81112222(-ve number)

LDR R1,=0x1112222(+ve number)

LDR R3,=0x0000

LDR R2,=0x0000

SMULL R2,R3,R0,R1

After Execution:

$R0 * R1 = 2\text{'s complement}(0x81112222) *$

$0x1112222$

$= 0x7EEEDDDE * 0x1112222$

$= 0x87\ 6DA7\ 2EEE\ F37C$ (-ve number)

Result $= 2\text{'s complement}(0x87\ 6DA7\ 2EEE\ F37C)$

$= 0xFF78\ 9258\ D111\ 0C84$

$R2 = 0x\ D111\ 0C84$ (Lower 32 bits)

$R3 = 0x\ FF78\ 9258$ (Upper 32 bits)

UDIV Instruction

- Performs unsigned division operation

LDR R0,=0x81112222

LDR R1,=0x2222

LDR R3,=0x0000

UDIV R3,R0,R1

After Execution:

R0=0x81112222

R1=0x2222

R3=0x0003C804(R0/R1)

LDR R0,=0x81112222

LDR R1,=0x0000

LDR R3,=0x0000

UDIV R3,R0,R1

After Execution:

R0=0x81112222

R1=0x0000

R3=0(R0/R1)

No divide by zero error is generated.
An integer divisions by zero returns zero.

SDIV Instruction

- Performs Signed division operation

LDR R0,=0x81112222(-ve number)

LDR R1,=0x2222(+ve number)

LDR R3,=0x0000

SDIV R3,R0,R1

After execution:

$R0/R1 = 2\text{'s complement}(0x81112222)/0x2222$

$= 0x7EEEDDDE/0x2222$

$= 0x\ 3B803$

$R3 = 2\text{'s complement}(0x\ 3B803)$

$R3 = 0xFFFC\ 47FD$

- **Signed Division rules**

- $+ \text{Dividend} / + \text{Divisor} = +Q \text{ and } +R$
- $+ \text{Dividend} / - \text{Divisor} = -Q \text{ and } +R$
- $- \text{Dividend} / + \text{Divisor} = -Q \text{ and } -R$
- $- \text{Dividend} / - \text{Divisor} = +Q \text{ and } -R$

- Identify the operation of following code snippet

LDR R0,=0x81112222(-ve number)

LDR R1,=0x90002222(-ve number)

LDR R3,=0x0000

SDIV R3,R0,R1

After execution:

R0/R1=2's complement(0x81112222)/
2's complement(0x90002222)
=0x7EEEDDDE/0x6FFF DDDE
=0x1

R3=0x1

- Identify the operation of following code snippet

LDR R0,=0x71112222(+ve number)

LDR R1,=0x90002222(-ve number)

LDR R3,=0x0000

SDIV R3,R0,R1

After execution:

R0/R1=0x71112222/
2's complement(0x90002222)
=0x71112222/0x6FFF DDDE
=0x1

R3=2's complement(0x1)

R3=0xFFFFFFFF

Logical Instructions

Instruction	Operation
AND Rd, Rn	$Rd = Rd \& Rn$
AND Rd, Rn, # immed	$Rd = Rn \& \#immed$
AND Rd, Rn, Rm	$Rd = Rn \& Rm$
ORR Rd, Rn	$Rd = Rd Rn$
ORR Rd, Rn, #immed	$Rd = Rn \#immed$
ORR Rd, Rn, Rm	$Rd = Rn Rm$
BIC Rd, Rn	$Rd = Rd \& (\sim Rn)$
BIC Rd, Rn, #immed	$Rd = Rn \& (\sim \#immed)$
BIC Rd, Rn, Rm	$Rd = Rn \& (\sim Rm)$

*No flags are updated because suffix 'S' is not there

* All instructions are conditional

Instruction	Operation
ORN Rd, Rn, # immed	$Rd = Rn \mid (\sim \#immed)$
ORN Rd, Rn, Rm	$Rd = Rn \mid (\sim Rm)$
EOR Rd,Rn	$Rd = Rd \wedge Rn$
EOR Rd,Rn, #immed	$Rd = Rn \wedge \#immed$
EOR Rd,Rn,Rm	$Rd = Rn \wedge Rm$

*No flags are updated because suffix 'S' is not there

* All instructions are conditional

- Identify the operation & working of following C program statements:

```
HAL_GPIO_WritePin(GPIOA,GPIO_PIN_2 | GPIO_PIN_3, GPIO_PIN_RESET);
```

```
HAL_GPIO_WritePin(GPIOA,GPIO_PIN_1 | GPIO_PIN_4 | GPIO_PIN_5 | GPIO_PIN_6 | GPIO_PIN_7, GPIO_PIN_SET);
```

Pin definitions:

```
#define GPIO_PIN_0      ((uint16_t) 0x0001)
#define GPIO_PIN_1      ((uint16_t) 0x0002)
#define GPIO_PIN_2      ((uint16_t) 0x0004)
#define GPIO_PIN_3      ((uint16_t) 0x0008)
#define GPIO_PIN_4      ((uint16_t) 0x0010)
#define GPIO_PIN_5      ((uint16_t) 0x0020)
#define GPIO_PIN_6      ((uint16_t) 0x0040)
#define GPIO_PIN_7      ((uint16_t) 0x0080)
#define GPIO_PIN_8      ((uint16_t) 0x0100)
#define GPIO_PIN_9      ((uint16_t) 0x0200)
#define GPIO_PIN_10     ((uint16_t) 0x0400)
#define GPIO_PIN_11     ((uint16_t) 0x0800)
#define GPIO_PIN_12     ((uint16_t) 0x1000)
#define GPIO_PIN_13     ((uint16_t) 0x2000)
#define GPIO_PIN_14     ((uint16_t) 0x4000)
#define GPIO_PIN_15     ((uint16_t) 0x8000)
#define GPIO_PIN_All    ((uint16_t) 0xFFFF)
```

```
void HAL_GPIO_WritePin(GPIO_TypeDef* GPIOx, uint16_t GPIO_Pin, GPIO_PinState PinState)
{
    if(PinState != GPIO_PIN_RESET)
    {
        GPIOx->BSRR = GPIO_Pin;
    }
    else
    {
        GPIOx->BSRR = (uint32_t)GPIO_Pin << 16U;
    }
}
```

BSRR

MGR

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
BR15	BR14	BR13	BR12	BR11	BR10	BR9	BR8	BR7	BR6	BR5	BR4	BR3	BR2	BR1	BR0
w	w	w	w	w	w	w	w	w	w	w	w	w	w	w	w
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
BS15	BS14	BS13	BS12	BS11	BS10	BS9	BS8	BS7	BS6	BS5	BS4	BS3	BS2	BS1	BS0
w	w	w	w	w	w	w	w	w	w	w	w	w	w	w	w

Bits 31:16 **BRy**: Port x reset bit y (y = 0..15)

These bits are write-only and can be accessed in word, half-word or byte mode. A read to these bits returns the value 0x0000.

0: No action on the corresponding ODRx bit

1: Resets the corresponding ODRx bit

Note: If both BSx and BRx are set, BSx has priority.

Bits 15:0 **BSy**: Port x set bit y (y= 0..15)

These bits are write-only and can be accessed in word, half-word or byte mode. A read to these bits returns the value 0x0000.

0: No action on the corresponding ODRx bit

1: Sets the corresponding ODRx bit

Use of logical operations:

- Note that ANDing a bit with 0 produces a 0 at the output while ANDing a bit with 1 produces the original bit.
- This can be used to create a **mask**.

Example:

1010 0011 1100 1111 1010	1110 1011 1101	LDR R0,=0xA3CF AEBD
0000 0000 0000 0000 0000	1111 1111 1111	LDR R1,=0x0000 0FFF

The result of ANDing these:

0000 0000 0000 0000 0000	1110 1011 1101	AND R2,R0,R1
--------------------------	----------------	--------------

Masking upper 16 bits

Use of logical operations:

- Similarly, note that **OR**ing a bit with 1 produces a 1 at the output while **OR**ing a bit with 0 produces the original bit. This can be used to force certain bits of a string to 1s.
- The **EOR** operation used to toggle the bits.

Example:

- $0x12345678 \text{ OR } 0x0000FFFF$ results in $0x1234FFFF$ (the high-order 16 bits are untouched, while the low-order 16 bits are forced to 1s).
- $0x0000\ 0078 \text{ EOR } 0x0000\ 00FF$ results in $0x0000\ 0017$.

Shift & Rotate Instructions

Instruction	Operation
ASR Rd, Rn, #immed	$Rd = Rn \gg \text{immed}$ (Arithmetic Shift)
ASR Rd, Rn	$Rd = Rd \gg Rn$
ASR Rd, Rn, Rm	$Rd = Rn \gg Rm$
LSL Rd, Rn, #immed	$Rd = Rn \ll \text{immed}$
LSL Rd, Rn	$Rd = Rd \ll Rn$
LSL Rd, Rn, Rm	$Rd = Rn \ll Rm$
LSR Rd, Rn, #immed	$Rd = Rn \gg \text{immed}$ (Logical Shift)
LSR Rd, Rn	$Rd = Rd \gg Rn$
LSR Rd, Rn, Rm	$Rd = Rn \gg Rm$

*Carry flags is updated if suffix 'S' is there

* All instructions are conditional

Shift & Rotate Instructions

Instruction	Operation
ROR Rd, Rn	Rd = Rd rotate by Rn
ROR Rd, Rn, Rm	Rd = Rn rotate by Rm
RRX Rd, Rn	RRX provides the value of the contents of a register shifted right one bit. The old carry flag is shifted into bit[31]. If the S suffix is present, the old bit[0] is placed in the carry flag. [C, Rd]=[Rn, C]

*Carry flags is updated if suffix 'S' is there

* All instructions are conditional

Identify the difference between following statements:

- LDR R1,=0x90002222
LDR R4,=0x0000
LSL R4, R1,#1

After execution:

R1=0x90002222

R4=0x20004444

No carry flag updated



LDR R1,=0x90002222

LDR R4,=0x0000

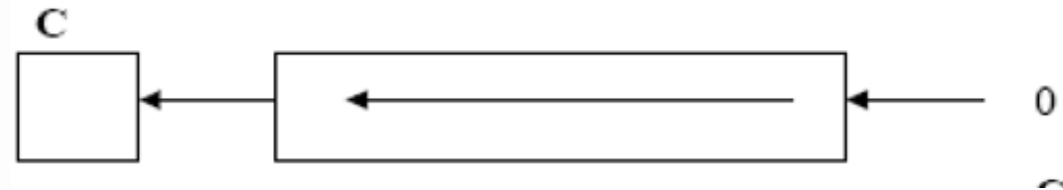
LSLS R4, R1,#1

After execution:

R1=0x90002222

R4=0x20004444

Carry flag=1



Shift Instructions...

Identify the difference between following statements:

LDR R1,=0x90002222

LDR R4,=0x0000

ASR R4, R1,#2

After execution:

R1=0x90002222

R4=0xE400 0888

No carry flag updated



LDR R1,=0x90002222

LDR R4,=0x0000

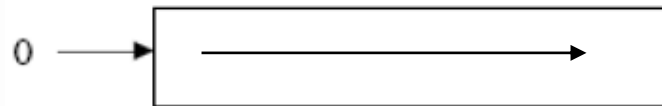
LSR R4, R1,#1

After execution:

R1=0x90002222

R4=0x2400 0888

No carry flag updated



LDR R1,=0x90002222

LDR R4,=0x0000

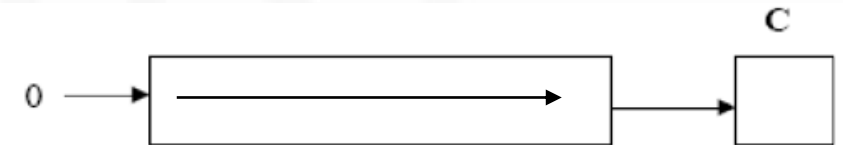
LSRS R4, R1,#1

After execution:

R1=0x90002222

R4=0x2400 0888

Carry flag=1



Shift Instructions...

Identify the difference between following code snippets:

```
LDR R1,=0x90002222
```

```
LDR R4,=0x0000
```

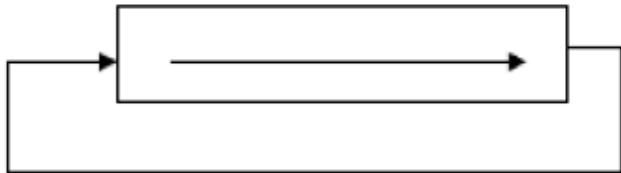
```
ROR R4, R1,#2
```

After execution:

R1=0x90002222

R4=0xA400 0888

No carry flag updated



```
LDR R1,=0x90002222
```

```
LDR R4,=0x0000
```

```
RORS R4, R1,#2
```

After execution:

R1=0x90002222

R4=0xA400 0888

Carry flag=1

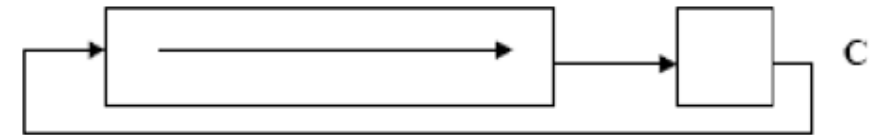


```
LDR R1,=0x90002222
```

```
RRX R0, R1
```

After execution:

R1=0x48001111



Instructions for Compare and Test

Instruction	Operation
CMP Rn, Rm	Compare: Calculate $Rn - Rm$. APSR is updated but the result is not stored.
CMP Rn, #immed	Compare: Calculate $Rn - \text{immediate data}$.
CMN Rn, Rm	Compare negative: Calculate $Rn + Rm$. APSR is updated but the result is not stored.
CMN Rn, #immed	Compare negative: Calculate $Rn + \text{immediate data}$. APSR is updated but the result is not stored.
TST Rn, Rm	Test (bitwise AND): Calculate AND result between Rn and Rm. N bit and Z bit in APSR are updated but the AND result is not stored.
TEQ Rn, #immed	Test (bitwise XOR): Calculate XOR result between Rn and immediate data. N bit and Z bit in APSR are updated but the AND result is not stored.

Program flow control instructions

Instruction set...

- **Unconditional Branch Instructions**

Instruction	Operation
B label B.W label	Branch to label. If a branch range of over $\pm 2\text{KB}$ is needed, you might need to specify B.W to use 32-bit version of branch instruction for wider range.
BX Rm	Branch and exchange. Branch to an address value stored in Rm.

- **Instructions for calling function**

Instruction	Operation
BL label	Branch to a labeled address and save the return address in LR
BLX Rm	Branch to an address specified by Rm, save the return address in LR.

Program flow control instructions

- Conditional Branch Instruction

Instruction	Operation
B<cond> <label>	Branch to label if condition is true

- Suffixes for Conditional Branches and Conditional Execution

Suffix	Branch Condition	Flags(APSR)
EQ	Equal	Z=1
NE	Not Equal	Z=0
CS/HS	Carry Set/Unsigned Higher or same	C=1
CC/LO	Carry clear / unsigned lower	C=0
MI	Minus	N=1
PL	Plus	N=0

- Conditional Branch Instruction

Suffix	Branch Condition	Flags(APSR)
VS	Overflow	V=1
VC	No Overflow	V=0
GE	Signed greater than or equal	N==V or Z=1
LE	Signed less than or equal	N!=V or Z=1
GT	Greater Than	N==V
LT	Less Than	N!=V

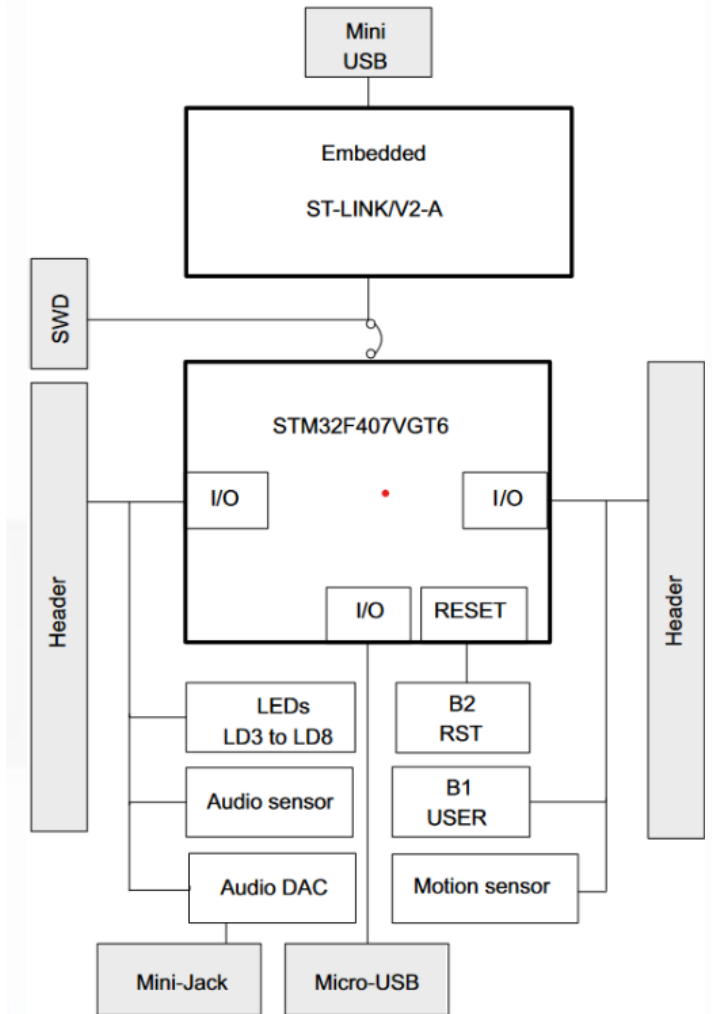
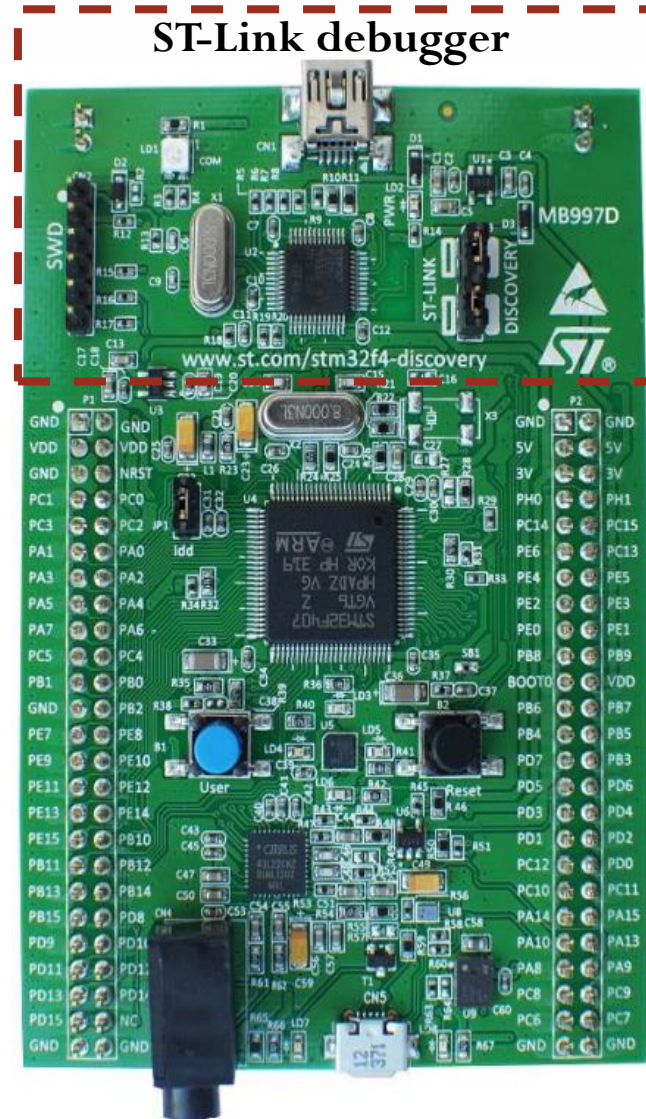
Integrated Development Environment(IDE)

- An IDE is a software application that provides comprehensive tools and features to facilitate software development. **E.g:Keil**
- **Code Editor:** A built-in text editor with syntax highlighting, code completion, and other features to help programmers write code more efficiently.
- **Compiler/Interpreter:** Compiler or interpreter to translate the written code into machine-readable instructions or execute it directly.
- **Assembler:** The assembler translates assembly language code into machine code, which is necessary for low-level programming and hardware-specific operations.
- **Linker:** The primary purpose of the linker is to take multiple object files generated during the compilation process and combine them into a single executable or library file that can be executed or used by other programs.

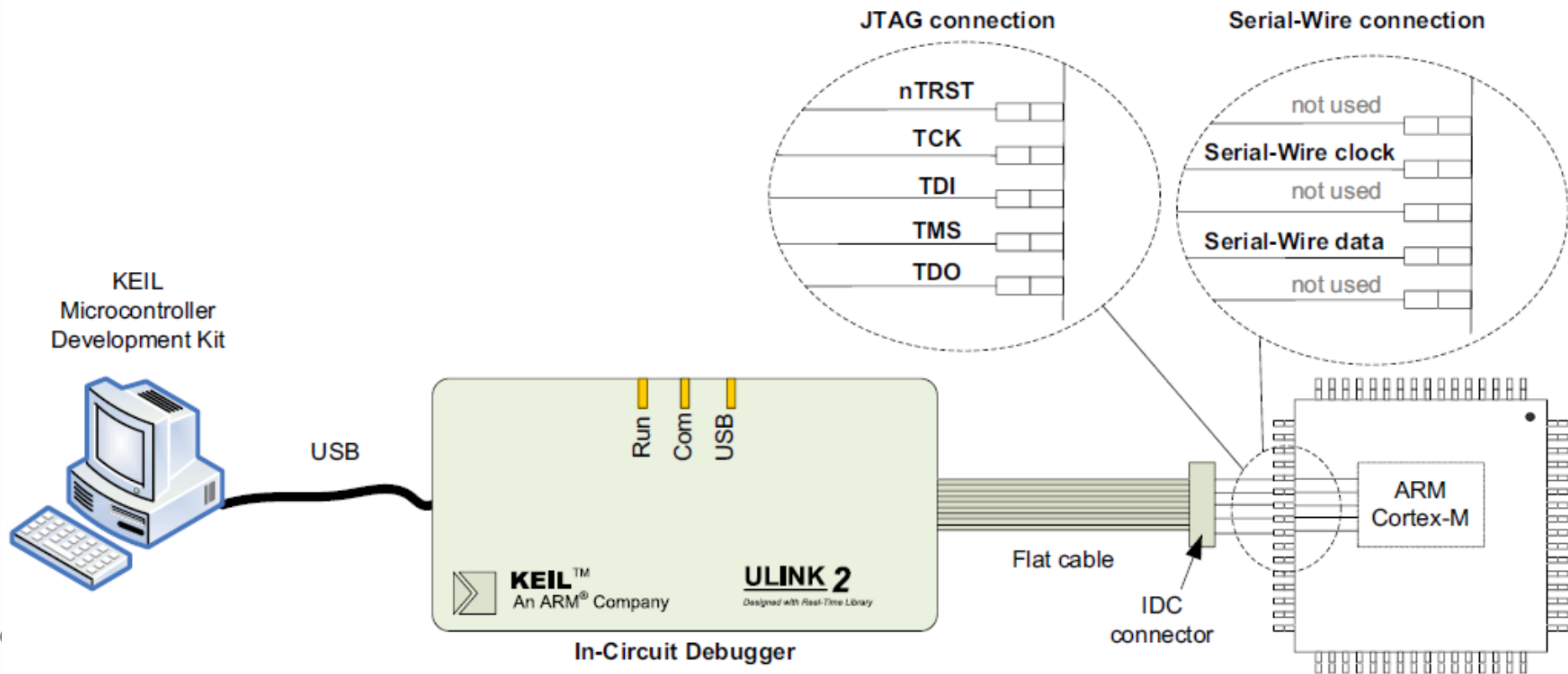
- **Debugger:** Debugging tools that allow developers to step through the code, set breakpoints, inspect variables, and identify and fix errors in the code.
- **Loader:** This is program or component responsible for loading an executable file into memory and preparing it for execution.
- **Simulation:** These capabilities, enabling us to test our code without running it on actual hardware.
- **Profiling Tools:** Some IDEs include performance profiling tools to help developers analyze resource utilization, execution time, etc.
- In addition, other tools will be there in IDEs.

ST-Link Debugger

- The ST-Link Debugger is a hardware debugging and programming tool developed by STMicroelectronics.
- The ST-Link Debugger allows developers to interact with STM32 microcontrollers during the development process, enabling them to program the microcontroller's flash memory, perform real-time debugging, and access hardware peripherals.
- The figure shows STM32F407VG discovery with ST-Link debugger.



- The debug Cortex-M processor supports the traditional JTAG(Joint Test Action Group) protocol, which uses either 4 or 5 pins, or a newer 2-pin protocol called Serial Wire Debug (SWD).
- ST-Link debugger makes use of SWD.
- Many commercially available debug adaptors, such as the ULINK 2 or ULINK Pro products from Keil support both protocols.



ST-Link Debugger...

- SWD has two pins: Serial Wire Clock(SWCLK) and Serial Wire Data Input Output(SWDIO).
- SWDIO is bidirectional and carries all the data and commands between the debugger and the target device.
- The SWCLK line synchronizes the communication between the debugger and the target device.
- SWD access the debug unit(refer slide 10) on ARM cortex M core which in turn communicate with other units to support debugging operation.
- The figure shows external ST-Link debuggers.

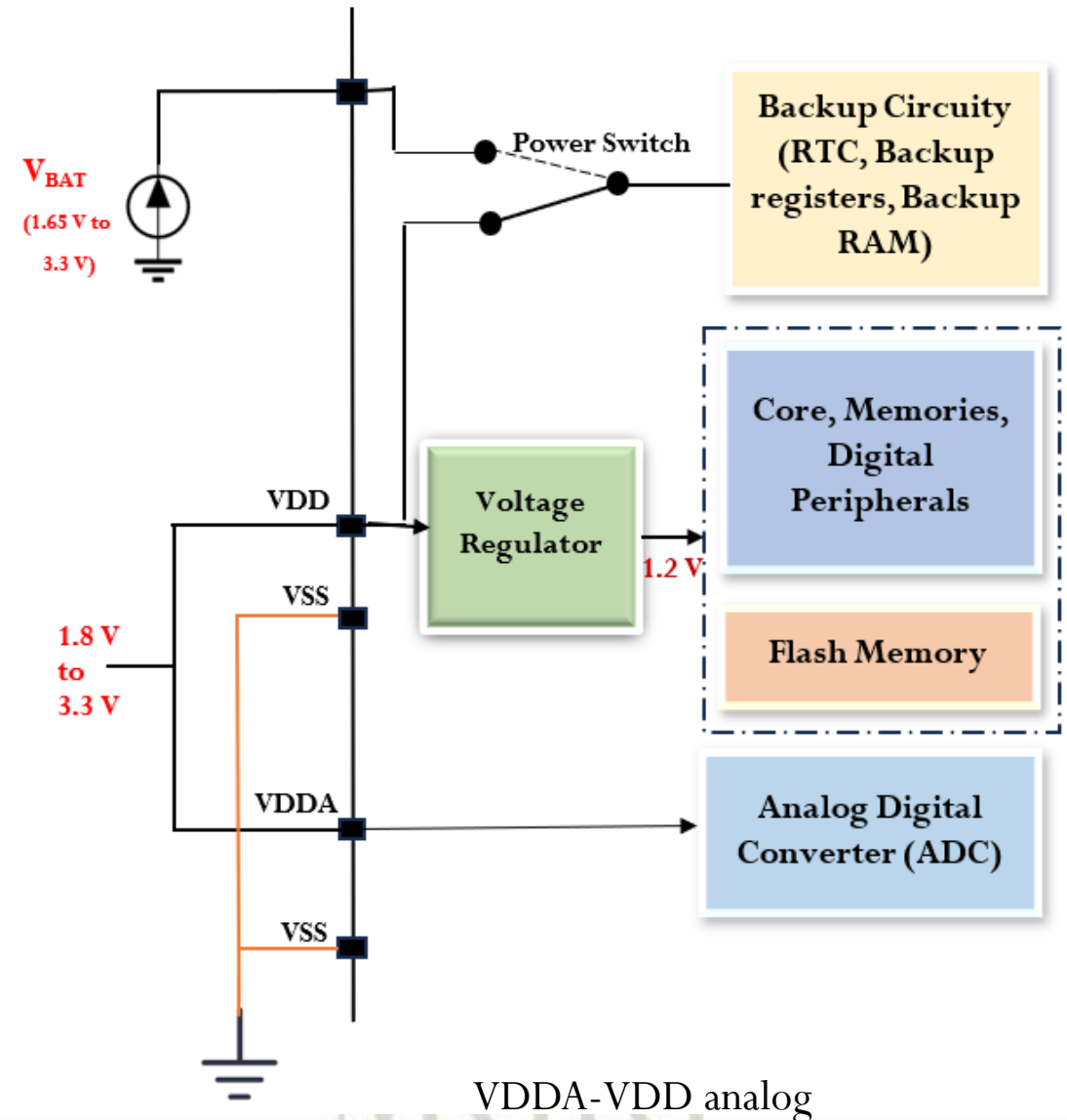


Low Power Modes

- By default, the microcontroller is in Run mode after a system or a power-on reset. In Run mode, program code is executed.
- Several low-power modes are available to save power when the CPU does not need to be kept running, for example when waiting for an external event.
- The STM32F407xx feature three low power modes:
 - Sleep mode (Cortex-M4 core stopped, peripherals kept running)
 - Stop mode (all clocks are stopped)
 - Standby mode (1.2 V domain powered off)
- In addition, the power consumption in Run mode can be reduce by one of the following means:
 - Slowing down the system clocks
 - Gating the clocks(turning off) to the APBx and AHBx peripherals when they are unused.

Power supplies in STM32F407xx

- To retain the content of the RTC backup registers, backup SRAM, and supply the RTC when VDD is turned off, V_{BAT} pin can be connected to an optional standby voltage supplied by a battery or by another source.
- An embedded linear voltage regulator supplies around 1.2V all the digital circuitries(Core, Memories, digital peripherals) except for the backup domain and the Standby circuitry.
- To improve conversion accuracy, the ADC has an independent power supply .



Low Power Modes....

Mode Name	Description	Effect on clocks	Voltage Regulator
Sleep	CPU CLK OFF No effect on other clocks or analog clock sources	No effect	ON
Stop	All 1.2V domain clocks are off	HSE(High Speed External) and HSI(High Speed Internal) clocks are off	ON or in Low power mode
Standby	All 1.2V domain clocks are off		OFF